

Sistema de Deteccion de Armas mediante Vision Artificial con YOLOv8

Documento tecnico del proceso de desarrollo

Plataforma de entrenamiento: NVIDIA RTX 5050 Laptop (Blackwell)

Plataforma de despliegue: Raspberry Pi 5

Fecha de generacion: 07/05/2026 12:19

Repositorio: deteccion-de-armas-i.a

Indice

1. Resumen ejecutivo
2. Objetivos del sistema
3. Arquitectura de hardware
4. Stack de software
5. Datasets y procesamiento de datos
6. Modelo y arquitectura YOLO
7. Configuración del entrenamiento
8. Fases ejecutadas
9. Resultados parciales
10. Despliegue en Raspberry Pi 5
11. Riesgos identificados y mitigaciones
12. Trabajo pendiente

1. Resumen ejecutivo

El presente documento describe el desarrollo de un sistema de detección en tiempo real de armas blancas (cuchillos) y armas de fuego (pistolas, escopetas, rifles) basado en la arquitectura YOLOv8 de Ultralytics. El sistema está diseñado para entrenarse en una laptop con GPU NVIDIA RTX 5050 y desplegarse en una Raspberry Pi 5 corriendo el modelo cuantizado a INT8 con ONNX Runtime.

Se ejecutaron dos iteraciones del proceso de entrenamiento. La primera versión (v1) utilizó únicamente Open Images v7 con dos clases agregadas (knife/firearm) y obtuvo un mAP@0.5 de 0.62 en el conjunto de prueba. La segunda versión (v2) incorpora datasets de Roboflow Universe, eleva el corpus a 8.656 imágenes curadas y descompone la clase firearm en handgun y long_gun para mejorar la coherencia visual.

2. Objetivos del sistema

- Detectar en tiempo real armas blancas (clase **knife**) y armas de fuego diferenciando armas cortas (**handgun**) de armas largas (**long_gun**).
- Lograr una velocidad de inferencia mínima de 10 FPS en Raspberry Pi 5 a resolución 416x416.
- Mantener el modelo desplegado en menos de 5 MB para permitir actualizaciones remotas eficientes.
- Disparar alertas locales (sonido, log y captura de evidencia) cuando se detecte una amenaza con confianza superior al umbral configurado.
- Permitir el despliegue completo en la Pi sin necesidad de instalar PyTorch ni Ultralytics, reduciendo la huella a menos de 250 MB de paquetes.

3. Arquitectura de hardware

3.1 Estacion de entrenamiento

Componente	Especificacion	Rol
Equipo	ASUS TUF Gaming A16 FA608UH	Entrenamiento + inferencia desktop
CPU	AMD Ryzen 7 260 (8 cores / 16 hilos)	DataLoader workers
RAM	32 GB DDR5	Cache de dataset
GPU dedicada	NVIDIA RTX 5050 Laptop, 8 GB VRAM	Entrenamiento CUDA
Arquitectura GPU	Blackwell, capability sm_120	Requiere CUDA 12.8+
GPU integrada	AMD Radeon 780M	Solo display, sin CUDA
Driver NVIDIA	581.80 (CUDA 13.0 runtime)	Detectado al iniciar

3.2 Plataforma de despliegue (objetivo)

Componente	Recomendado
SBC	Raspberry Pi 5 (8 GB RAM)
Almacenamiento	microSD 64 GB clase A2
Refrigeracion	Active cooler oficial (imprescindible)
Camara	USB UVC o Pi Camera v3
Sistema operativo	Raspberry Pi OS Bookworm 64-bit
Alimentacion	Fuente oficial 5V/5A

4. Stack de software

El entorno se aísla en un virtualenv Python 3.11.9 dentro del repositorio (repo/.venv). Se forzó la asignación de la GPU NVIDIA para los ejecutables Python vía la clave de registro HKCU\Software\Microsoft\DirectX\UserGpuPreferences.

Paquete	Version	Funcion
torch	2.11.0+cu128	Framework de deep learning con soporte CUDA 12.8
torchvision	0.26.0+cu128	Operaciones de vision en PyTorch
ultralytics	8.4.47	Framework YOLOv8 (entrenamiento + export)
fiftyone	1.15.0	Gestion del dataset Open Images v7
roboflow	1.x	Descarga de datasets Roboflow Universe
opencv-python	4.13.0	Captura de camara y dibujo de bboxes
onnx + onnxruntime	1.21 / 1.25	Export e inferencia portable
onnxslim	0.1.92	Optimizacion del grafo ONNX
pygame	2.6.1	Reproduccion de alertas sonoras
reportlab	4.5.0	Generacion del presente documento

5. Datasets y procesamiento de datos

5.1 Fuentes utilizadas

Fuente	Imagenes	Clases originales	Mapeo final
Open Images v7 (FiftyOne)	2.185	Knife, Handgun, Shotgun, Rifle	knife / handgun / long_gun
RF joseph-nelson/pistols	2.971	pistol	handgun
RF rifledetection/rifle-detection-p9slr	3.500*	Rifle	long_gun

* Sampleadas aleatoriamente desde un total de 13.013 imagenes para mantener balance de clases.

5.2 Reorganizacion taxonomica

La version v1 utilizaba una clase generica **firearm** que agrupaba pistolas, escopetas y rifles. Esta decision degrado el rendimiento en esa clase a un mAP@0.5 de 0.49 debido a la alta varianza visual entre los subtipos. Para v2 se establecio la siguiente taxonomia final:

- **0 — knife:** cuchillos y armas blancas en general.
- **1 — handgun:** armas cortas (pistolas, revolveres). Empuniadura a una mano.
- **2 — long_gun:** armas largas (rifles, escopetas). Requieren dos manos.

5.3 Particion final

Particion	Imagenes	Anotaciones knife	Anotaciones handgun	Anotaciones long_gun
Train (85%)	7.357	983	3.593	5.363
Val (10%)	865	93	413	699
Test (5%)	434	51	217	322
Total	8.656	1.127	4.223	6.384

5.4 Formato YOLO

Cada imagen NNNNNNN.jpg tiene asociado un archivo NNNNNNN.txt con una linea por objeto detectado:

```
<class_id> <cx> <cy> <ancho> <alto>
```

Donde todas las coordenadas estan normalizadas en el rango [0, 1] respecto al tamaño de la imagen, y (cx, cy) es el centro del bounding box.

6. Modelo y arquitectura YOLO

YOLO (You Only Look Once) es una familia de detectores de objetos en una sola pasada. A diferencia de R-CNN o Fast R-CNN que primero proponen regiones candidatas y luego las clasifican, YOLO predice simultaneamente clase y bounding box procesando la imagen una unica vez. Esto la convierte en la opcion adecuada para inferencia en tiempo real en hardware limitado como la Raspberry Pi.

6.1 Variantes evaluadas

Variante	Parametros	Tamano (FP32)	Uso en este proyecto
YOLOv8n (nano)	3.2 M	~6 MB	Despliegue final en Pi5
YOLOv8s (small)	11 M	~22 MB / 89 MB con optimizer	Entrenamiento en laptop
YOLOv8m (medium)	26 M	~52 MB	No utilizado (excede VRAM)

6.2 Estrategia dual de modelos

Se entrena YOLOv8s en la laptop como modelo de referencia con la mejor precision posible. Posteriormente se entrena un YOLOv8n con el mismo dataset, optimizado para tamano y velocidad, que es el que se exporta a ONNX cuantizado para la Pi.

Se utiliza fine-tuning desde pesos preentrenados en COCO (yolo`v8s.pt`, `yolov8n.pt`). Esto reduce drasticamente el tiempo de entrenamiento al partir de un modelo que ya conoce características visuales generales (bordes, texturas, formas).

7. Configuracion del entrenamiento

7.1 Hiperparametros principales

Parametro	Valor	Justificacion
epochs	100	Suficiente con dataset ampliado
batch	8	Reducido desde 12 para evitar OOM al final
workers	4	Reducido desde 8 para liberar RAM
imgsz	640	Estandar YOLOv8 en GPU
optimizer	AdamW	Adaptativo, robusto, estandar moderno
lr0	0.001	Conservador para fine-tuning
patience	30	Early stopping si val no mejora
amp	true	FP16 mixed precision (mitad de VRAM)
device	0	GPU CUDA 0 (RTX 5050)
seed	42	Reproducibilidad

7.2 Augmentations aplicadas

Augmentation	Valor	Efecto
fliplr	0.5	50% prob. espejo horizontal
flipud	0.0	Sin espejo vertical (irreal)
mosaic	1.0	Combina 4 imagenes en una
mixup	0.15	Transparenta dos imagenes
copy_paste	0.3	Copia objetos entre imagenes (clave para knife)
hsv_h/s/v	0.015 / 0.7 / 0.4	Variacion de tono, saturacion, brillo

7.3 Variables de entorno de seguridad

```
PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True
CUDA_VISIBLE_DEVICES=0
KMP_DUPLICATE_LIB_OK=TRUE
```

`expandable_segments:True` es la mitigacion clave del incidente de OOM observado en la version v1 (epoch 70/80). Permite a PyTorch expandir segmentos de VRAM existentes en lugar de fragmentar la memoria con asignaciones nuevas.

8. Fases ejecutadas

Fase	Nombre	Descripcion	Estado
Fase 0	Setup del entorno	Instalacion de Python 3.11.9, creacion del virtualenv, instalacion de PyTorch 2.11+cu128 (verificado <code>torch.cuda.is_available()==True</code> , device RTX 5050 Laptop GPU sm_120), verificacion de matmul CUDA, instalacion de dependencias del requirements.txt. Configuracion de preferencia de GPU NVIDIA en el registro de Windows.	Completado
Fase 1	Descarga inicial de OpenImages v7	Primera ejecucion de <code>1_download_dataset.py</code> . Se identificaron dos bugs: (1) la clase 'Firearm' no existe como leaf class en OIv7; (2) el parametro <code>dataset_dir</code> entraba en conflicto con argumentos internos de FiftyOne. Ambos corregidos, descarga exitosa de 2.185 imagenes en 4 clases.	Completado
Fase 2	Conversion a formato YOLOv8	Mapeo OIv7 -> 2 clases (knife/firearm), generacion de splits y data.yaml. 1.500 imgs train + 169 val + 516 test = 2.185 imagenes con 3.612 anotaciones validas.	Completado
Fase 3	Entrenamiento YOLOv8s	100 epochs configuradas, batch 12, workers 8. Crash con CUDA OOM en epoch 70/80 por fragmentacion de VRAM. <code>best.pt</code> se conservo (mAP esperado ya alcanzado por early stopping efectivo).	Completado con incidente
Fase 4	Evaluacion v1	Sobre el split de test (516 imgs): <code>mAP@0.5 = 0.617</code> , <code>mAP@0.5:0.95 = 0.474</code> , Precision = 0.83, Recall = 0.65. Velocidad de inferencia: 4.4 ms por imagen (equivalente a 227 FPS en RTX). Identificado bajo rendimiento en clase firearm.	Completado
Fase 5	Plan de mejora y obtencion de Stable Roboflow	Se establecio estrategia: ampliar el corpus con datasets curados de Roboflow Universe, descomponer firearm en handgun + long_gun, ajustar config para evitar OOM, planificar export INT8 para Pi.	Completado
Fase 6	Descarga de datasets Roboflow	Se identificaron y descargaron <code>joseph-nelson/pistols</code> (2.971 imgs) y <code>rifledetection/rifle-detection-p9slr</code> (13.013 imgs, sampleadas a 3.500). Total descargado: ~1.8 GB.	Completado
Fase 7	Merge y unificacion del dataset	Implementacion de <code>2b_merge_datasets.py</code> : re-mapeo de clases a 3 categorias, particion estratificada 85/10/5, validacion de imagenes corruptas. Resultado: 8.656 imagenes finales.	Completado

Fase 8	Reentrenamiento YOLOv8n	Canvado con batch 8, workers 4, 100 epochs, patience 30, augmentations extendidas (copy_paste 0.3) y env vars de proteccion VRAM. En curso.	En progreso
Fase 9	Entrenamiento YOLOv8n	Pendiente de iniciar tras finalizar v2. Mismo dataset, imgsz 416, batch 16, 80 epochs.	Pendiente
Fase 10	Export ONNX INT8	Cuantizacion INT8 con calibracion sobre 100 imgs del train. Salida esperada: best.onnx ~3 MB.	Pendiente
Fase 11	Evaluacion final y deploy	Reporte de mAP por clase del modelo cuantizado, copia a Pi via scp, configuracion de servicio systemd para arranque automatico.	Pendiente

9. Resultados parciales

9.1 Comparacion v1 vs v2

Metrica	v1 (2 clases, Olv7)	v2 (3 clases, Olv7+RF)
Imagenes de entrenamiento	1.500	7.357
Total dataset	2.185	8.656
mAP@0.5 (test)	0.617	En progreso (>0.71 en val a epoch 19)
mAP@0.5:0.95 (test)	0.474	En progreso (0.48 en val a epoch 19)
Precision	0.83	0.71 actual (val)
Recall	0.65	0.67 actual (val)
Inferencia laptop	4.4 ms/img (227 FPS)	Esperado similar

9.2 Curva de aprendizaje v2

Hasta epoch 19 (al momento de generacion de este documento), el modelo v2 muestra una curva de aprendizaje ascendente y sin signos de overfitting:

Epoch	mAP@0.5 (val)	mAP@0.5:0.95 (val)
1	0.443	0.246
2	0.514	0.293
3	0.557	0.345
6	0.628	0.396
12	0.681	0.431
19	0.719	0.479

9.3 Uso de recursos durante v2

- VRAM utilizada: 2.71 GB de 8 GB disponibles (33%).
- Temperatura GPU sostenida: 77 C (limite: 87 C).
- Utilizacion GPU: 86% promedio.
- Tiempo por epoch: aproximadamente 115 segundos.
- RAM total reportada por SO: 93%, pero pagefile usado solo 439 MB (memoria standby reclamable).

10. Despliegue en Raspberry Pi 5

10.1 Software a instalar en la Pi

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y python3-pip python3-venv libatlas-base-dev libopenblas-dev
python3 -m venv ~/weapons-env
source ~/weapons-env/bin/activate
pip install opencv-python onnxruntime numpy pyyaml pygame
```

Total instalado: aproximadamente 250 MB. **No** se instala PyTorch ni Ultralytics.

10.2 Archivos a copiar desde la laptop

Archivo	Funcion	Tamano aprox.
models/export/best.onnx	Modelo cuantizado INT8	~3 MB
config.yaml	Umbrales y configuracion	2 KB
utils/	Visualizacion + alertas	10 KB
scripts/7_inference_pi.py	Loop de inferencia	5 KB

Total a transferir: **aproximadamente 3 MB**.

10.3 Performance esperado

Resolucion	Modelo	FPS esperados en Pi 5
416x416	YOLOv8n FP32 ONNX	4 - 7
416x416	YOLOv8n INT8 ONNX	12 - 18 (objetivo)
320x320	YOLOv8n INT8 ONNX	18 - 25
416x416	YOLOv8n NCNN FP16	15 - 22

10.4 Servicio systemd para arranque automatico

```
[Unit]
Description=Weapons Detection
After=network.target

[Service]
User=pi
WorkingDirectory=/home/pi/weapons
ExecStart=/home/pi/weapons-env/bin/python 7_inference_pi.py --weights best.onnx
Restart=always

[Install]
WantedBy=multi-user.target
```

11. Riesgos identificados y mitigaciones

Riesgo	Impacto	Mitigacion aplicada
OOM de VRAM al final del entrenamiento (en época 70/80, perdida de progreso de entrenamiento)	Crash en época 70/80, perdida de progreso de entrenamiento	Red de progreso de entrenamiento, workers a 4, env var expandable_segments
Fragmentacion progresiva de VRAM	OOM progresivo tras horas de entrenamiento	BYE_FORCE_CUDA_ALLOC_CONF configurada
Clase minoritaria (knife) con solo 12 imágenes	Baja precisión	copy_paste augmentation 0.3 + datasets adicionales
TDR de Windows durante carga GPU	Flash de negro momentaneo	ocultar el contenido de GPU y USB
GPU NVIDIA Blackwell sm_120 incompatible con CUDA 12.1	Crash al iniciar	PyTorch instaló CUDA 12.0 en la iau128 explicitamente
Falta de assets/alert.wav en el repo	Crash al activar alertas sonoras	assets.sound_enabled puesto en false hasta proveer el archivo

12. Trabajo pendiente

- Finalizar entrenamiento YOLOv8s v2 (en curso, estimado 2-3 horas).
- Entrenar YOLOv8n con el mismo dataset para despliegue en Pi.
- Exportar YOLOv8n a ONNX e INT8 con calibracion sobre 100 imgs.
- Evaluar el modelo cuantizado en el split de test, verificar perdida de mAP respecto al FP32 (esperada < 2%).
- Generar archivo de alerta sonora assets/alert.wav.
- Aprovisionar Raspberry Pi 5 con sistema operativo y dependencias.
- Realizar pruebas de campo con camara en la Pi y medir FPS reales.
- Considerar incorporacion de hard negatives (200-500 imagenes sin armas con objetos confusos como utensilios de cocina) para reducir falsos positivos.

Documento generado automaticamente. Las cifras de la version v2 corresponden al estado del entrenamiento al momento de la generacion y se actualizaran al completarse el run.